What is Shell in Linux?

- A Shell is a program that provides the interface between the user and the Operating System.
- It allows users to give commands to the system in human-readable form, and then translates them into machine instructions.
- The shell acts as an interpreter.
- When you log in, a default shell starts automatically.
- It loads startup files to set environment variables, command paths, and aliases.
- The following are the different types of shells :
  - Bash (Bourne Again Shell) ,
  - sh (Bourne shell), explain only main points
  - csh (C shell),
  - ksh (Korn shell),
  - zsh

Shell Responsibilities

1. Program Execution: Shell runs all the commands you type.

Example: ls -l // Shell runs the ls program with -l option.

Format: program-name arguments

2. Handling Whitespace & Arguments:

Shell ignores extra spaces.

Example: $ echo when do we eat?

→ Shell passes four arguments (when, do, we, eat?) to echo.

Output: when do we eat?

3. Variable and Filename Substitution

You can assign values to variables and use them.

name=John
echo $name → John

4. Supports wildcards like *, ? for filenames.

Example: ls s* → Lists all files starting with s.

5. I/O Redirection: Shell can redirect input/output.

Example: echo Hello > file.txt
→ Saves “Hello” into file.txt (instead of showing on screen).

6. Pipeline Hookup

Shell can connect multiple commands with | (pipe).
Example: who | sort | lpr
→ who lists users → sort arranges → lpr sends to printer.

7. Interpreted Programming Language

Shell itself is a programming language.

Supports loops, conditions, variables, and functions.

1. Making Scripts Executable

When we write a shell script (a text file with commands), the system does not automatically allow it to be executed. We must give it execute permissions.This is done using the chmod command.

Example: chmod 744 script_file
chmod u+x script_file

2. Executing the Script

Once executable, the script can be run in two main ways:

(a) Independent Command Execution

If the script has the interpreter designator line (also called shebang) at the top, like:

#!/bin/bash

then you can run it directly:

./script_name

Here ./ tells the shell to look in the current directory.

(b) Child Shell Execution

Instead of executing directly, we can explicitly call a shell to run the script:

bash script_name
sh script_name

This way, the script runs inside a new shell (child shell).

Here, the shebang (#!/bin/bash) is not mandatory, but the user must know which shell to use (bash, sh, zsh, etc.).

Drawback: If you don't specify the correct shell, the script may not run properly.
Best Practice: Always include the shebang (#!/bin/bash) at the top of the script.

The Shell as a Programming Language

- A **shell script** is a text file that contains executable commands.
- Although we can execute virtually any command at the shell prompt, long sets of commands that are going to be executed more than once should be executed using script file

Creating a Script

- To create a shell script first use a text editor to create a file containing the commands.

Script Components

- ***Interpreter Designator Line***
- The first line of the script is the interpreter designator line.
- It tells LINUX the path to the appropriate shell interpreter.
- The designator line begins with a pound sign and a bang (#!).
- If it is omitted ,Linux uses current shell's default interpreter.

#!/bin/bash or #!/usr/bin/bash

The Shell as a Programming Language

Comments

- Comments are documentation we add in a script to help us understand it.
- The interpreter doesn't use the at all; it simply skips over them.
- Comments are identified with the pound sign token (#).

Recommended minimum documentation

SCRIPT

```
1. #!/bin/bash                    # Bourne again shell path name
  2. #   Script: doc.scr
  3. # This script demonstrates our recommended minimum documentation
  4.
  5. # Commands Start here
```

The Shell as a Programming Language

Commands

- The most important part of the script is its commands.
- We can use any of the commands available in UNIX.

Example for Shell script

Myfile.scr

```
#!/bin/bash
# Script: Myscript.scr
#Extracting lines from a file

head  -47 myfile |  tail  +42 > newfile.txt
```

Execute the above script

```
$  myscript.scr
$ cat newfile.txt
```

The Shell as a Programming Language

Script Termination (exit command)

- There are 3 streams or standard files. The shell sets up these 3 standard files and attaches them to user terminal at the time of logging in.
- Standard i/p ----default source is the keyboard.
- Standard o/p ----default source is the terminal.
- Standard error ----default source is the terminal.

Each of the standard files has a number called a file descriptor, which is used for identification.

0—standard i/p

1---standard o/p

2---standard error

Shell Metacharacters

- These are special characters that are recognized by the shell

`*` This is a 'wildcard', it matches any string of zero or more characters, except a leading '.' An example of its use:

```
% ls *.mod
```

This will list all the files in the current directory that end with a `.mod`

```
% ls ca*
```

This will list all the files that commence with the letters 'ca' and have zero or more characters afterwards.

`?` This will match any single character An example of its use:

```
% ls tut?.m
```

This will find files such as `tuta.m`, `tut1.m`, etc.

`[ccc]` This will match any single character from the string *ccc*. Ranges such as 0–9, a–z, and A–Z are also valid. An example of its use:

```
% ls tut[0-9].m
```

This will find files such as `tut1.m`, `tut2.m`, etc

```
% ls *.[ch]
```

This will list all files in the current directory that end in either `.c` or `.h`.

File Name Substitution

- File name substitution is a feature which allows special characters and patterns to substituted with file names in the current directory, or arguments to the **case** and [[...]] commands.

?	match any single character
*	match zero or more characters, including null
[**abc**]	match any characters between the brackets
[**x–z**]	match any characters in the range **x** to **z**
[**a–ce–g**]	match any characters in the range **a** to **c**, **e** to **g**
[**!abc**]	match any characters not between the brackets
[**!x–z**]	match any characters not in the range **x** to **z**

Shell Variables

- A variable is a location in memory where values can be stored.
- A variable can store or manipulate the information within the shell program.
- Shell allows us to create, store and access values in variables
- Each shell variable must have a name .

Rules to declare variable

- The name of a variable must start with an alphanumeric or underscore(_) character.
- It then can be followed by zero or more alphanumeric or underscore characters.
- Two types of shell variables: **User-defined and Predefined**

User-Defined Variables

- User variables are not separately defined UNIX.
- A shell variable can be used for assigning a value and accessing the value .

Storing Data in Variables

Assignment: *Variable_name* = *Value* {using assignment operator '='}

Reference: $Variable_name

Example : $x=23

```
$ echo $x
  23
```

Shell Variables

Accessing a Variable

• To access the value of a variable, the name of the variable must be preceded
by a dollar sign.

• We use **echo** command to display the values.

Example:

```
$x=23
$ echo The variable x contains $x
 The variable x contains 23.
```

To Remove a Variable

• Unset command is used to remove a variable from the shell.

Example:

```
$ unset x
$ echo $x
$
```

Assignment of Multiple Word Strings to a Variable

• Assigning multiple words to a variable is done by writing it in single quotes.

Example:

```
$message='you have been short listed';
echo $message     #You have been short listed
```

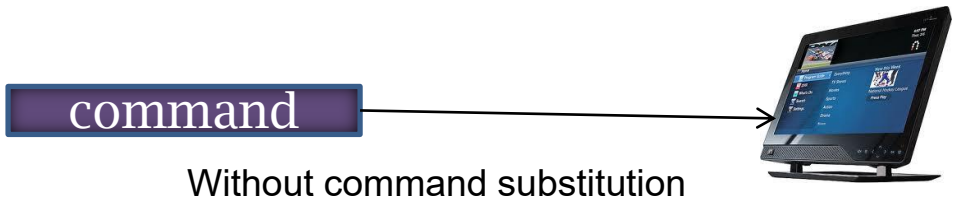
Shell Variables

Special shell variables

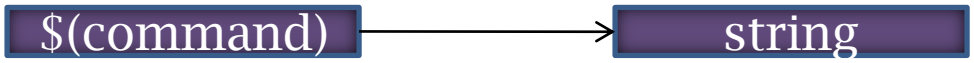
Parameter	Meaning
$0	Name of the current shell script
$1-$9	Positional parameters 1 through 9
$#	The number of positional parameters
$*	All positional parameters, “$*” is one string
$@	All positional parameters, “$@” is a set of strings
$?	Return status of most recently executed command
$$	Process id of current process

Command Substitution

- When a Shell executes a command, the output is directed to standard output(most of the time, standard output is associated with monitor. **Ex: date**
- Shell allows the standard output of one command to be used as an argument of another command **EX: echo "Today is $(date)"**
- Some times we need to change the output to a string that we can store in another string or a variable .**EX: mydate=$(date)**
- Command substitutions provides the capability to convert the result of a command to a string . The command substitution operator that converts the output of a command to a string is a dollar sign and a set of parentheses. **EX: $(command)**



Without command substitution



With command substitution

Shell Commands

- The commands used inside the shell for programming tasks are known as the shell commands, those are:

1) expr command:

- The expr utility is used to evaluate arithmetic operations, as shell doesn't support arithmetic operators.
- Expression will be evaluated by expr utility and the result is sent to the standard output.
- Special symbols such as *,>,< must be preceded by a \, otherwise the shell treats it as a meta-character and yields wrong results

Example:

```
i=5
j=3
echo `expr $i + $j`   # 8
echo `expr $i - $j`   # 2
echo `expr $i \* $j`  # 15
echo `expr $i / $j`   # 1
echo `expr $i % $j`   # 2
```

Shell Commands

2) who | sort

- who command displays who logged onto the system and provides an account of all users
- It displays a three column output, the first column displays the user-ids of users currently working on the system.
- The second column displays the system name.
- The third column displays the date and time.
- Sort command sorts text files. It sorts all the lines from the standard input.
- who | sort before displaying the who output on the screen it is piped to sort.
- Sort receives the output of who as standard input, it the sorts the output according to the first alphabet in each line and displays it on screen

Shell Commands

3) ls | wc –l

- ls command lists all the files and directories on the standard output.
- wc –l command counts the number of files it receives as standard input
- ls command is piped to wc –l command where the standard output of ls is given as standard input to wc coomand.
- Thus, it displays the count of files and directories on the standard output

Shell Commands

4) break

- Break statement causes the control to come out of the loop instantly.
- It terminates the loop.

```
# break.sh
#!/bin/sh
x=5
while [ $x -gt 3 ]          # outer while loop
do
   y=5
   while [ $y -gt 2 ]       # inner while loop
   do
      if [ $y -eq 3 ]
      then
         break              # immediately terminates inner loop when y=3
      else
         echo $x $y
      fi
      y=`expr $y - 1`
   done
   x=`expr $x - 1`
done
```

Shell Commands

6) Aliases

- An alias provides a means of creating customized commands by assigning a name to a command.
- An alias is created by using the alias command. Its format is

alias name=command-definition

Example :

Using alias to rename the list command

```
$ alias dir=ls
$ dir
```

alias of command with options

```
$alias dir='ls –l'
$ dir
```

Listing aliases

```
$alias
```

Removing aliases

```
$alias dir
$unalias dir
```

Removing all aliases

```
$unalias -a
```

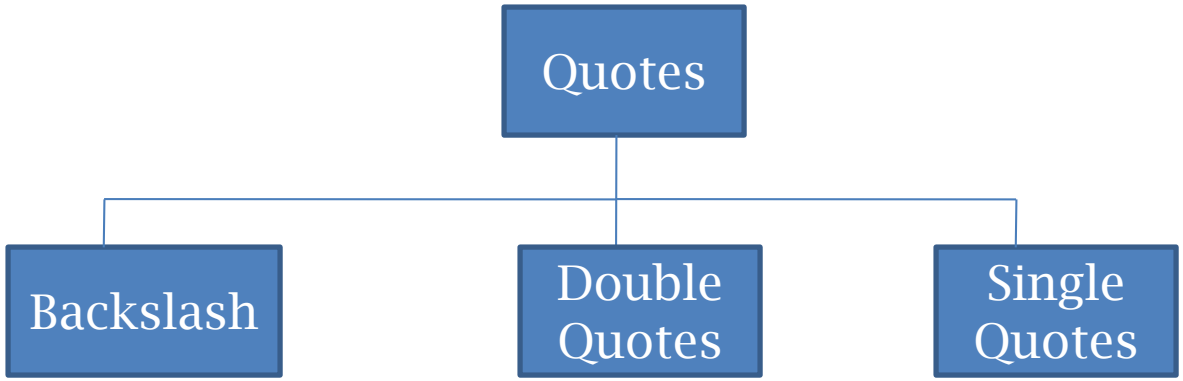
The Environment

Environmental Variables

- Environmental variables control the user environment.
- Environmental variables are set by the environment, the operating system, during the startup of your shell.
- Reading environment variables same as shell variables.
  For example :
  an environment variable named HOME, you can access the value of this variable by using the dollar($) sign, for example $HOME
- The environmental variables are in uppercase.
- Commonly used environmental variables are:
- CDPATH, HISTFILE,
- HOME, LOGNAME,

Quoting

- Shell uses a selected set of metacharcters in commands.
- Metacharcters are characters that have a special interpretation.
- For example the pipe (|) .
- In order to use them as a normal text we must tell the shell interpreter that they should be interpreted differently.
- This is called quoting the metacharcters.
- To achieve quoting there are three metacharcters collectively as quotes,



Quoting

Backslash

- The backslash metacharcter (\) is used to change the interpretation of the character that follows it, i.e., it converts a literal character into a special character and vice versa.

Example:

- The character n is interpreted as a literal character by the shell to change its interpretation as a newline character we use backslash before it (\n).
- Similarly the use of a greater than symbol (>) in a command is interpreted as a special character (output redirection), to change its interpretation as a literal text we use a backslash before it (\>)

```
$echo My name is - \> Ramesh\n
  My name is ->Ramesh
```

Quoting

Double Quotes

- Double quotes (“) are used to change the meaning of several characters.
- They change the special interpretation of most metacharcters like <,>,?,& and so on.
- These metacharcters are treated as literal characters.

Example:

```
$ echo “Metacharcters are : >,<,?,|,&“
Metacharcters are : >,<,?,|,&
```

- Double quotes cannot remove the special interpretation of a dollar sign in front of a variable and single quotes.

```
$ echo a = 10
$ echo “a is $a and single quote is ‘b’ “
   a is 10 and single quote is ‘b’
```

Quoting

Single Quotes

- Like double quotes, single quotes change the special interpretation of metacharcters.
- It not only treats metacharcters like >,<,?,$ and so on as literals but also the metacharcters dollar sign ($) and every double quotes.

Example:

```
$ echo a = 10
$ echo ‘characters are < > “b” $a ? &’
 characters are < > “b” $a ? &
```

TEST Command

- The test command is used to check file types and compare values. Test is used in conditional execution. It is used for:
- Numeric comparison
- File attributes comparisons
- Perform string comparisons.

test command syntax

test condition
OR
test condition **&&** true-command
OR
test condition **||** false-command
OR
test condition **&&** true-command **||** false-command

Type the following command at a shell prompt (is 5 greater than 2?):

test 5 -gt 2 **&& echo** "Yes"
test 1 -lt 2 **&& echo** "Yes“

Sample Output:
Yes
Yes

TEST Command

Numeric comparison

- Numerical tests are implied when comparison between values of two numbers is to be done.
- Different numerical operators are :

Numerical Comparison Operators Used by test

Operator	Syntax	Description
-eq	INTEGER1 -eq INTEGER2	INTEGER1 is equal to INTEGER2
-ge	INTEGER1 -ge INTEGER2	INTEGER1 is greater than or equal to INTEGER2
-gt	INTEGER1 -gt INTEGER2	INTEGER1 is greater than INTEGER2
-le	INTEGER1 -le INTEGER2	INTEGER1 is less than or equal to INTEGER2
-lt	INTEGER1 -lt INTEGER2	INTEGER1 is less than INTEGER2
-ne	INTEGER1 -ne INTEGER2	INTEGER1 is not equal to INTEGER2

TEST Command

String Comparisons

- String comparison can be done using test command itself.
-

Condition	Description
Str1 = Str2	True if str1 is equal to str2
Str1 != Str2	True if str1 is not equal to str2
Strg	Ture if string strg is not null and is assigned
-n strg	True if string strg is not a null (or empty string)
-z strg	True if string strg is a null (or empty) string
Str1 == str2	True if string str1 is equal to str2

TEST Command

File attributes comparisons

• It checks all the file attributes. It checks whether a file is an ordinary file, or a directory, or it has read, write or executable permissions

Option	Description
-f file	True if file exists and is a regular file
-r file	True if file exists and having a read permission
-w file	True if file exists and have a write permission
-x file	True if file exists and have a executable permission
-d file	True if file exists and is a directory
-s file	True if file exists and has a size greater than zero
-e file	True if file exists
File1 –nt file2	True if file1 is newer than file2
File1 –ot file2	True if file1 is older than file2
File1 –ef file2	True if file1 is linked to file2

Control Structures

- Control structures alter the flow of the program

- Control Structures are: 1) conditional statements 2) control statements

1. if
2. if –else
3. Nested if
4. case

Control Statements:

1. loops
    a. for
    b. While
    c. Until

2. Handling of signals.

Control Structures

- **THE SIMPLE IF STATEMENT**

```
if [ condition ]; then
   statements
fi
```

- Executes the statements only if **`condition`** is true

Control Structures

- **THE IF-THEN-ELSE STATEMENT**

```
if [ condition ]; then
        statements-1
else
  statements-2
fi
```

- executes statements-1 if condition is true
- executes statements-2 if condition is false

Control Structures

- **THE NESTED IF**

```
if [ condition ]; then
     statements
elif [ condition ]; then
    statement
else
    statements
fi
```

- The word **`elif`** stands for “else if”
- It is part of the if statement and cannot be used by itself

Control Structures

THE CASE STATEMENT

• use the case statement for a decision that is based on multiple choices

Syntax:

```
case word in
        pattern1) command-list1
        ;;
        pattern2) command-list2
        ;;
        patternN) command-listN
        ;;
esac
```

case pattern

• checked against word for match

may also contain:

```
*
?
[ … ]
[:class:]
```

multiple patterns can be listed via:

```
|
```

Control Structures

THE WHILE LOOP

• Purpose:

To execute commands in “command-list” as long as “expression” evaluates to true

Syntax:

```
while [ expression ]
do
        command-list
done
```

Control Structures

THE UNTIL LOOP

• Purpose:

To execute commands in “command-list” as long as “expression” evaluates to false

Syntax:

```
until [ expression ]
do
        command-list
done
```

Control Structures

THE FOR LOOP

• Purpose:

To execute commands as many times as the number of words in the "argument-list"

Syntax:

```
for variable in argument-list
do
        commands
done
```

Control Structures

Signals on Linux

```
% kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL
 5) SIGTRAP      6) SIGABRT      7) SIGBUS       8) SIGFPE
 9) SIGKILL     10) SIGUSR1     11) SIGSEGV     12) SIGUSR2
13) SIGPIPE     14) SIGALRM     15) SIGTERM     16) SIGSTKFLT
17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU
25) SIGXFSZ     26) SIGVTALRM   27) SIGPROF     28) SIGWINCH
29) SIGIO       30) SIGPWR      31) SIGSYS      34) SIGRTMIN
35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3  38) SIGRTMIN+4
39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12
47) SIGRTMIN+13 48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14
51) SIGRTMAX-13 52) SIGRTMAX-12 53) SIGRTMAX-11 54) SIGRTMAX-10
55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7  58) SIGRTMAX-6
59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

^C is 2 - SIGINT

Arithmetic in Shell

Shell Arithmetic

• Use to perform arithmetic operations.

Syntax:
expr op1 math-operator op2

Examples:
$ expr 1 + 3
$ expr 2 - 1
$ expr 10 / 2
$ expr 20 % 3
$ expr 10 * 3
$ echo `expr 6 + 3`
Note:
expr 20 %3 - Remainder read as 20 mod 3 and remainder is 2.
expr 10 * 3 - Multiplication use * and not * since its wild card.

Functions

- A shell function is similar to a shell script
  - stores a series of commands for execution later
  - shell stores functions in memory
  - shell executes a shell function in the same shell that called it
- Where to define
  - In .profile
  - In your script
  - Or on the command line
- Remove a function
  - Use unset built-in

Functions

- must be defined before they can be referenced
- usually placed at the beginning of the script

Syntax:

```
function-name () {
    statements
}
```

Functions

Function parameters

- Need not be declared
- Arguments provided via function call are accessible inside function as $1, $2, $3, …

- $# reflects number of parameters
- $0 still contains name of script
  (not name of function)

Functions

Local Variables in Functions

- Variables defined within functions are global, i.e. their values are known throughout the entire shell program

- Keyword “local” inside a function definition makes referenced variables “local” to that function

Debugging Shell Scripts

- Debugging is troubleshooting errors that may occur during the execution of a program/script
- The following two commands can help you debug a bash shell script:
  - echo

    use explicit output statements to trace execution
  - set

Debugging Shell Scripts

Debugging using "set"

- The "set" command is a shell built-in command
- has options to allow flow of execution
  - –v option prints each line as it is read
  - –x option displays the command and its arguments
  - –n checks for syntax errors
- options can turned on or off
  - To turn on the option: set -xv
  - To turn off the options: set +xv

- Options can also be set via she-bang line

#! /bin/bash -xv